# CAD Project 3

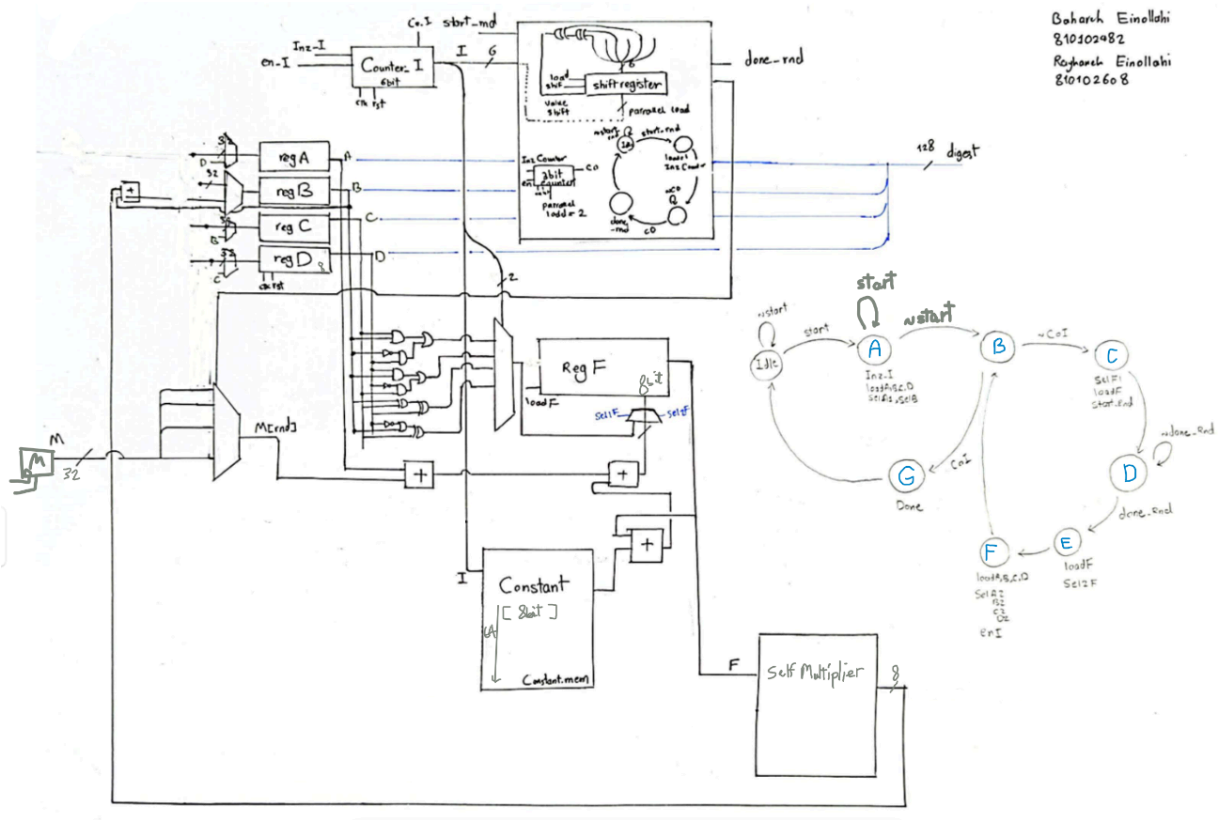
Report and design specification

Rayhaneh Einollahi - 810102608

Bahareh Einollahi - 810102482

بهار 1404

## Date path and controller:



## Base Gates Implementation:

For lowering number of the gates for combination circuit, we tried to use c1 modules and avoid c2.
The gates are all defined with c1:

```
module logic_not (input a, output out);
    c1 c13(.A0(1'b1) , .A1(1'b0) , .SA(a) , .B0(1'b0) , .B1(1'b0) ,
.SB(1'b0) , .S0(1'b0) , .S1(1'b0),.f(out)) ;

endmodule

module logic_and2 (input a, input b, output out);
    c1 c13(.A0(1'b0) , .A1(b) , .SA(a) , .B0(1'b0) , .B1(1'b0) , .SB(1'b0)
, .S0(1'b0) , .S1(1'b0),.f(out)) ;

endmodule

module logic_and3 (input a, input b, input c, output out);
    c1 c13(.A0(1'b0) , .A1(1'b0) , .SA(1'b0) , .B0(1'b0) , .B1(b) , .SB(a)
, .S0(c) , .S1(c),.f(out)) ;

endmodule

module logic_or2 (input a, input b, output out);
    c1 c13(.A0(b) , .A1(1'b1) , .SA(a) , .B0(1'b0) , .B1(1'b0) , .SB(1'b0)
, .S0(1'b0) , .S1(1'b0),.f(out)) ;

endmodule

module logic_xor2 (input a, input b, output out);
    c1 c13(.A0(a) , .A1(a) , .SA(a) , .B0(1'b1) , .B1(1'b0) , .SB(a) ,
.S0(b) , .S1(b),.f(out)) ;

endmodule
```

A basic dff implementation:

```
module logic_dff (input clk, input rst, input d, output out);
    s2 s2_dff (
        .D00(d), .D01(1'b0), .D10(1'b0), .D11(1'b0),
        .A1(1'b0), .B1(1'b0), .A0(1'b0), .B0(1'b0),
        .clr(rst), .clk(clk), .out(out));
endmodule
```

This special type of mux is used to for clarity:

When sel1 is high “a” is selected when sel2 is high “b” is selected and when non is high “otherwise” is selected.

```
module logic_mux (input sel_1, input a, input sel_2, input b, input
otherwise, output out);
        c1 c13(.A0(otherwise) , .A1(b) , .SA(sel_2) , .B0(otherwise) ,
.B1(a) , .SB(sel_1) , .S0(sel_1) , .S1(sel_1),.f(out)) ;

endmodule
```

## Base Modules Implementation:

### Parallel_in_mux module:

A combination of 8 logic_mux described above for parrallel data selection:

```
module parallel_in_mux
#(parameter WIDTH = 8)
(
    input sel_1,
    input  [WIDTH-1:0] a,
    input sel_2,
    input  [WIDTH-1:0] b,
    input  [WIDTH-1:0] otherwise,
    output [WIDTH-1:0] out
);
    genvar i;
    generate
        for (i = 0; i < WIDTH; i = i + 1) begin : bit_slice_mux
            logic_mux mux_i (
                .sel_1(sel_1),
                .a(a[i]),
                .sel_2(sel_2),
                .b(b[i]),
                .otherwise(otherwise[i]),
                .out(out[i])
            );
        end
    endgenerate
endmodule
```

## Full adder logic:

For minimum number of gates the logic was implemented in this form:

```
module full_adder (input a, input b, input cin, output sum, output cout);
    wire cin_inv;
    logic_not not1(cin, cin_inv);
    //b ^ a ^ cin
    c1 c13(.A0(cin) , .A1(cin_inv) , .SA(a) , .B0(cin_inv) , .B1(cin) ,
.SB(a) , .S0(b) , .S1(b),.f(sum)) ;
    c1 c24(.A0(1'b0) , .A1(cin) , .SA(b) , .B0(cin) , .B1(1'b1) , .SB(b) ,
.S0(a) , .S1(a),.f(cout)) ;
endmodule
```

The sum is the xor of a , b and cin.
First with a basic c1 module the invert of cin was generated,
Then the upper mux calcaultes: cin ^ a
The lower mux calculus : ~(cin^a)

And the last mux that selected between two by value of b gives the result : a ^ b ^ cin

For Cout:
The upper mux calculates : cin & b -> if a is zero then both cin and b have to be one if we eant to have a carry out
The lower calultes : cin | b : if “a” is one then if one of b and cin is one then we would have a carry out.

## Other adders logic:

Specific adders for the begin and end of a adding array was designed to minimize gates:

For the start where there is no cin:

```
module half_adder(input a, input b, output sum, output cout);
    logic_xor2 c1 (.a(a), .b(b), .out(sum));
    logic_and2 c2 (.a(a), .b(b), .out(cout));
endmodule
```

For the end where the carry out is not necessary:

```
module full_adder_no_cout(input a, input b, input cin, output sum);
    wire cin_inv;
    logic_not not1(cin, cin_inv);
    c1 c1_sum(.A0(cin) , .A1(cin_inv) , .SA(a), .B0(cin_inv) , .B1(cin) , .SB(a), .S0(b) , .S1(b),.f(sum));
endmodule
```

### 8bit adder with the logic described:

```
module adder
#(parameter WIDTH = 8)
(   input  [WIDTH-1:0] a,
    input  [WIDTH-1:0] b,
    output [WIDTH-1:0] sum);
    wire [WIDTH:0] c;
    assign c[0] = 1'b0;
    genvar i;
    generate
        for (i = 0; i < WIDTH; i = i + 1) begin : adder_slice
            if (i == 0) begin
                half_adder ha (
                    .a(a[i]),
                    .b(b[i]),
                    .sum(sum[i]),
                    .cout(c[i+1])
                );
            end
            else if (i == WIDTH - 1) begin
                full_adder_no_cout fa_last (
                    .a(a[i]),
                    .b(b[i]),
                    .cin(c[i]),
                    .sum(sum[i])
                );
            end else begin
                full_adder fa (
                    .a(a[i]),
                    .b(b[i]),
                    .cin(c[i]),
                    .sum(sum[i]),
                    .cout(c[i+1])
                );
            end
        end
    endgenerate
Endmodule
```

### 8bit adder with the logic described:

This is logic that generate the f next value,

```
module build_f1( input a, input b, input c, input d, input [1:0]sel,
output out);
    wire [3:0] res;
    wire out_res0_res1;
    wire d_inv;
    logic_not not1(d, d_inv);
    //(b & c) | (~b & d)
    //(d & b) | (~d & c)
    c1 c12(.A0(d) , .A1(c) , .SA(b) , .B0(c) , .B1(b) , .SB(d) ,
.S0(sel[0]) , .S1(sel[0]), .f(out_res0_res1)) ;
    //b ^ c ^ d
    c1 c13(.A0(d) , .A1(d_inv) , .SA(c) , .B0(d_inv) , .B1(d) , .SB(c) ,
.S0(b) , .S1(b),.f(res[2])) ;
    //c ^ (b | ~d)
    c2 c24(.D00(1'b0), .D01(1'b1), .D10(1'b1), .D11(1'b0) , .A1(b) ,
.B1(d_inv) , .A0(c) , .B0(c) , .out(res[3]));
    c2 c25(.D00(out_res0_res1), .D01(out_res0_res1), .D10(res[2]),
.D11(res[3]) , .A1(sel[1]) , .B1(sel[1]) , .A0(sel[0]) , .B0(sel[0]) ,
.out(out));
endmodule
```

For efficiency the first line of logic is a simple select between d and c by the value of b,
And the second line is another select between b,c and d. We used a single c1 for both of them and used sel[0] for early selecting which one will be used.

The next line is a xor of three that is explained in full adder.
And then next line used c2 and to compline all used c25 to select between thoes.

### This module for insitaing 8 of the above module:

```
module parallel_build_f1 (input [7:0] a,input [7:0] b,input [7:0] c,input [7:0] d,input
[1:0] sel, output [7:0] out);
    genvar i;
    generate
        for (i = 0; i < 8; i = i + 1) begin : f1_gen
            build_f1 f1_unit ( .a(a[i]), .b(b[i]), .c(c[i]), .d(d[i]), .sel(sel),
.out(out[i]));
        end
    endgenerate
Endmodule
```

### build_m_rnd:

```
module build_m_rnd (
    input [31:0] out_m,
    input [1:0]  rnd_out,
    output [7:0] m_rnd_
);
genvar k;
generate
    for (k = 0; k < 8; k = k + 1) begin : mux_gen
        c1 c13(.A0(out_m[31-k]) , .A1(out_m[23-k]) , .SA(rnd_out[0]) ,
.B0(out_m[15-k]) , .B1(out_m[7-k]) , .SB(rnd_out[0]) , .S0(rnd_out[1]) ,
.S1(rnd_out[1]),.f(m_rnd_[7-k])) ;
    end
endgenerate
endmodule
```

### Counter:

The carry out are calculated with an and gate and the new value is calculated in sum:

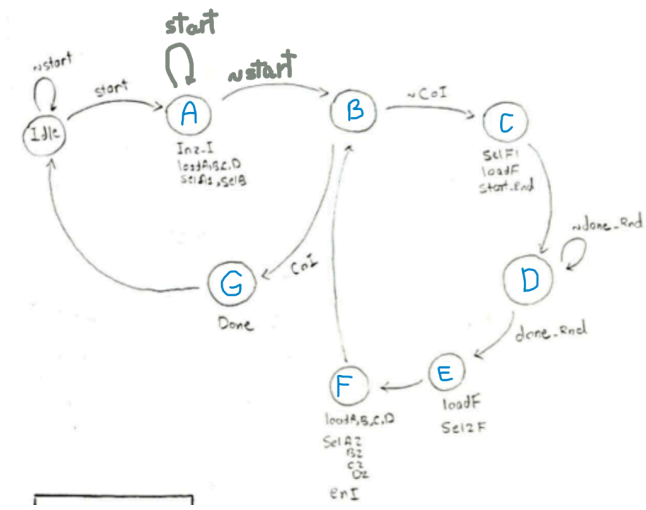
```
module counter #(parameter WIDTH = 3)
( input clk, rst, inz, en, input [WIDTH-1:0] inz_value, output cout, output cout2, output
[WIDTH-1:0] out);
    wire [WIDTH-1:0] sum;
    wire [WIDTH-1:0] carry;
    genvar i;
    generate
        for (i = 0; i < WIDTH; i = i + 1) begin : logic_chain
            if (i == 0) begin
                assign carry[i] = out[i];
            end else begin
                logic_and2 and_gate (out[i], carry[i-1], carry[i]);
            end
        end
    endgenerate

    assign cout = carry[WIDTH-1];
    generate
        for (i = 0; i < WIDTH; i = i + 1) begin : sum_logic
            if (i == 0) begin
                logic_not not_gate (out[i], sum[i]);
            end else begin
                logic_xor2 xor_gate (out[i], carry[i-1], sum[i]);
            end
        end
    endgenerate

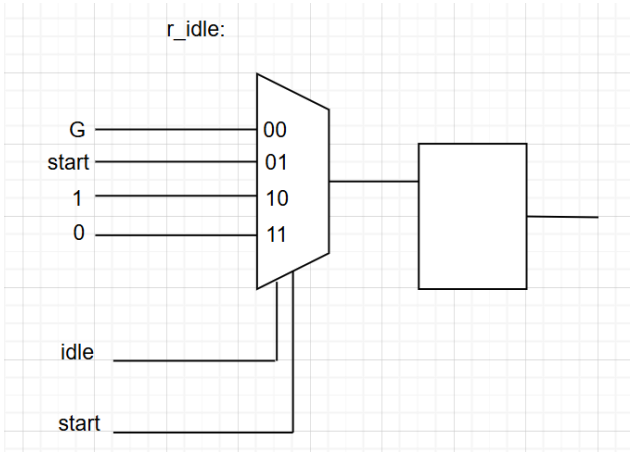
    generate
```

```
        for (i = 0; i < WIDTH; i = i + 1) begin : registers
            s2 bit_reg (.D00(out[i]), .D01(sum[i]), .D10(inz_value[i]), .D11(inz_value[i]),
.A1(inz), .B1(1'b0), .A0(en),  .B0(1'b1), .clr(rst), .clk(clk), .out(out[i]));
        end
    endgenerate
    s2 cout2_reg ( .D00(cout2), .D01(cout), .D10(1'b0), .D11(1'b0), .A1(inz), .B1(1'b0),
.A0(en), .B0(1'b1), .clr(rst), .clk(clk), .out(cout2));
endmodule
```

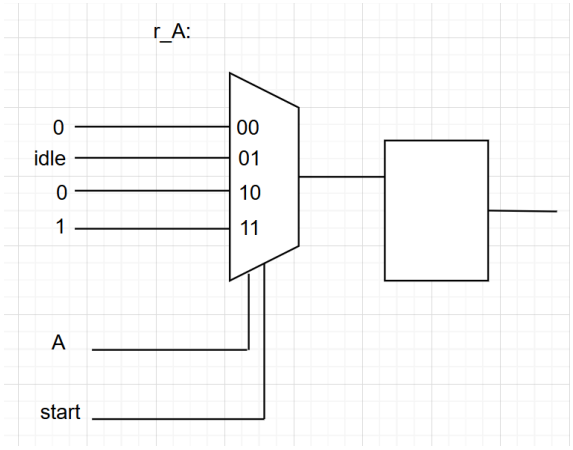
## Controller Implementation:




The design of idle:



The design of r_A register:




```
`timescale 1ns/1ns

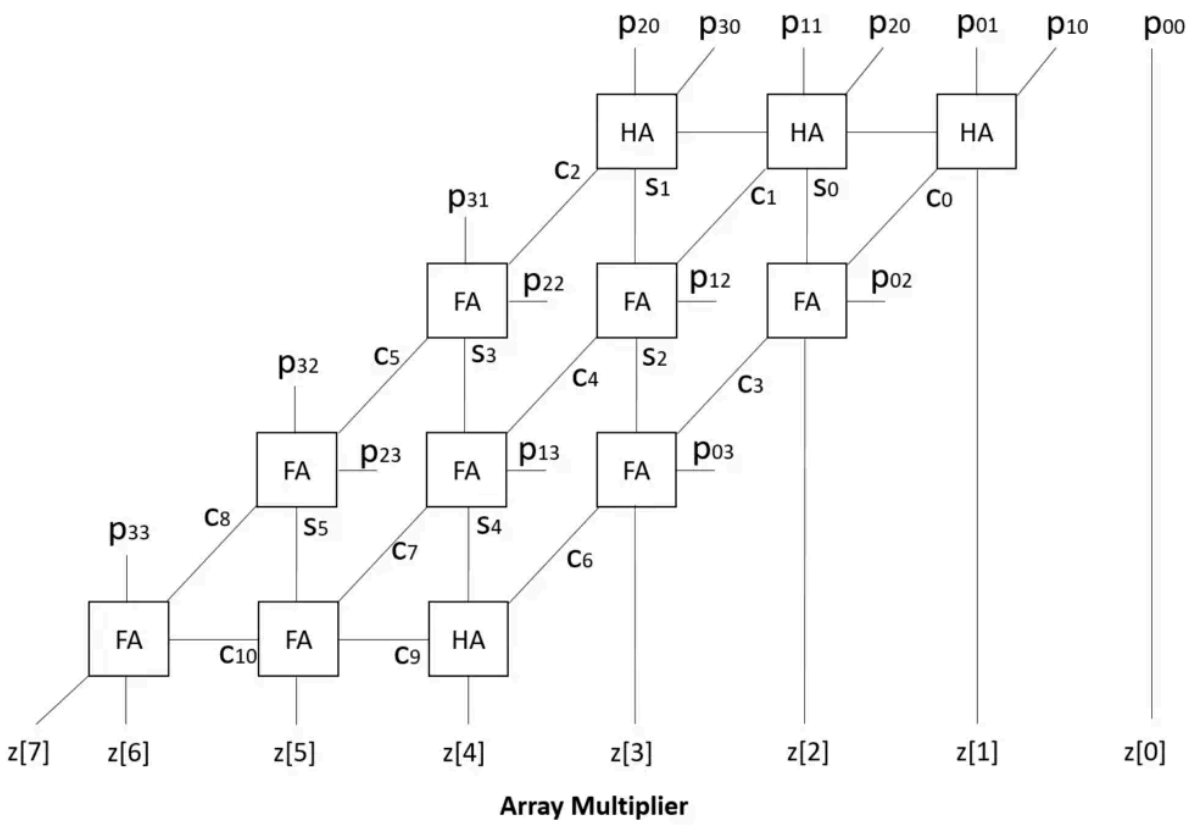
module hash_ct(
    ...

    s2 r_idle ( .D00(qg), .D01(start), .D10(1'b1), .D11(1'b0),
.A1(q_idle), .B1(1'b0), .A0(start), .B0(1'b1), .clr(rst), .clk(clk),
.out(q_idle));
    s2 r_a ( .D00(1'b0), .D01(q_idle), .D10(1'b0), .D11(1'b1), .A1(qa),
.B1(1'b0), .A0(start), .B0(1'b1), .clr(rst), .clk(clk), .out(qa));
    s2 r_b ( .D00(qf), .D01(qf), .D10(1'b1), .D11(1'b0), .A1(qa),
.B1(1'b0), .A0(start), .B0(1'b1), .clr(rst), .clk(clk), .out(qb));
    s2 r_c ( .D00(1'b0), .D01(1'b0), .D10(1'b1), .D11(1'b0), .A1(qb),
.B1(1'b0), .A0(co_counter_i), .B0(1'b1), .clr(rst), .clk(clk), .out(qc));
    s2 r_d ( .D00(qc), .D01(qc), .D10(1'b1), .D11(1'b0), .A1(qd),
.B1(1'b0), .A0(done_rnd), .B0(1'b1), .clr(rst), .clk(clk), .out(qd));
    s2 r_e ( .D00(1'b0), .D01(1'b0), .D10(1'b0), .D11(1'b1), .A1(qd),
.B1(1'b0), .A0(done_rnd), .B0(1'b1), .clr(rst), .clk(clk), .out(qe));
    s1 r_f ( .D00(1'b0), .D01(1'b0), .D10(1'b1), .D11(1'b1), .A1(qe),
.B1(1'b0), .A0(1'b0), .clr(rst), .clk(clk), .out(qf));
    s2 r_g ( .D00(1'b0), .D01(1'b0), .D10(1'b0), .D11(1'b1), .A1(qb),
.B1(1'b0), .A0(co_counter_i), .B0(1'b1), .clr(rst), .clk(clk), .out(qg));

    logic_or2 or_common (qa, qf, w_load_common);
    logic_or2 or_load_f (qc, qe, w_load_f);

endmodule
```

## Multiplier Implementation:




**Array Multiplier**

For the boundary full adders half_adders are used for optimization:

```
//----------------------------------------------------------------
// number of gates:                                        520
//----------------------------------------------------------------

module multiplier(
    input [7:0] f,
    output [7:0] multiplier_out
);
    wire [3:0] a;
    wire [3:0] b;
    assign a = f[7:4];
    assign b = f[3:0];

    wire pp0_0, pp0_1, pp0_2, pp0_3;
    wire pp1_0, pp1_1, pp1_2, pp1_3;
    wire pp2_0, pp2_1, pp2_2, pp2_3;
    wire pp3_0, pp3_1, pp3_2, pp3_3;
```

```
    logic_and2 pa00 (.a(a[0]), .b(b[0]), .out(pp0_0));
    logic_and2 pa01 (.a(a[1]), .b(b[0]), .out(pp0_1));
    logic_and2 pa02 (.a(a[2]), .b(b[0]), .out(pp0_2));
    logic_and2 pa03 (.a(a[3]), .b(b[0]), .out(pp0_3));

    logic_and2 pa10 (.a(a[0]), .b(b[1]), .out(pp1_0));
    logic_and2 pa11 (.a(a[1]), .b(b[1]), .out(pp1_1));
    logic_and2 pa12 (.a(a[2]), .b(b[1]), .out(pp1_2));
    logic_and2 pa13 (.a(a[3]), .b(b[1]), .out(pp1_3));

    logic_and2 pa20 (.a(a[0]), .b(b[2]), .out(pp2_0));
    logic_and2 pa21 (.a(a[1]), .b(b[2]), .out(pp2_1));
    logic_and2 pa22 (.a(a[2]), .b(b[2]), .out(pp2_2));
    logic_and2 pa23 (.a(a[3]), .b(b[2]), .out(pp2_3));

    logic_and2 pa30 (.a(a[0]), .b(b[3]), .out(pp3_0));
    logic_and2 pa31 (.a(a[1]), .b(b[3]), .out(pp3_1));
    logic_and2 pa32 (.a(a[2]), .b(b[3]), .out(pp3_2));
    logic_and2 pa33 (.a(a[3]), .b(b[3]), .out(pp3_3));

    wire s1_0, c1_0, s1_1, c1_1, s1_2, c1_2, s1_3, c1_3;
    wire s2_0, c2_0, s2_1, c2_1, s2_2, c2_2, s2_3, c2_3;
    wire s3_0, c3_0, s3_1, c3_1, s3_2, c3_2, s3_3, c3_3;

    assign multiplier_out[0] = pp0_0;

    half_adder ha1_0 (.a(pp0_1), .b(pp1_0), .sum(s1_0), .cout(c1_0));
    full_adder fa1_1 (.a(pp0_2), .b(pp1_1), .cin(c1_0), .sum(s1_1), .cout(c1_1));
    full_adder fa1_2 (.a(pp0_3), .b(pp1_2), .cin(c1_1), .sum(s1_2), .cout(c1_2));
    half_adder ha1_3 (.a(pp1_3), .b(c1_2), .sum(s1_3), .cout(c1_3));

    assign multiplier_out[1] = s1_0;

    half_adder ha2_0 (.a(s1_1), .b(pp2_0), .sum(s2_0), .cout(c2_0));
    full_adder fa2_1 (.a(s1_2), .b(pp2_1), .cin(c2_0), .sum(s2_1), .cout(c2_1));
    full_adder fa2_2 (.a(s1_3), .b(pp2_2), .cin(c2_1), .sum(s2_2), .cout(c2_2));
    full_adder fa2_3 (.a(c1_3), .b(pp2_3), .cin(c2_2), .sum(s2_3), .cout(c2_3));

    assign multiplier_out[2] = s2_0;

    half_adder ha3_0 (.a(s2_1), .b(pp3_0), .sum(s3_0), .cout(c3_0));
    full_adder fa3_1 (.a(s2_2), .b(pp3_1), .cin(c3_0), .sum(s3_1), .cout(c3_1));
    full_adder fa3_2 (.a(s2_3), .b(pp3_2), .cin(c3_1), .sum(s3_2), .cout(c3_2));
    full_adder fa3_3 (.a(c2_3), .b(pp3_3), .cin(c3_2), .sum(s3_3), .cout(c3_3));

    assign multiplier_out[3] = s3_0;
    assign multiplier_out[4] = s3_1;
    assign multiplier_out[5] = s3_2;
    assign multiplier_out[6] = s3_3;
    assign multiplier_out[7] = c3_3;

endmodule
```

## Rnd generator controller Implementation:

The number of states are low so a simple controller with two registers for four states were used:

```
module rnd_generator_ct(
    input clk,
    input rst,
    input start_rnd,
    output wire shift_shreg,
    output wire load_shreg,
    output wire done_rnd
);
    wire inz_counter, en_counter, co_counter;
    wire q1, q0;
    counter counter( .clk(clk), .rst(rst), .inz(inz_counter),
.en(en_counter), .inz_value(3'b010), .cout(co_counter));
    s2 state_reg_q1 (.D00(1'b0), .D01(1'b1), .D10(1'b1), .D11(1'b0),
.A1(q1), .B1(1'b0), .A0(q0), .B0(1'b1), .clr(rst), .clk(clk), .out(q1));
    s2 state_reg_q0 ( .D00(start_rnd), .D01(1'b0), .D10(co_counter),
.D11(1'b0), .A1(q1),.B1(1'b0), .A0(q0),.B0(1'b1), .clr(rst), .clk(clk),
.out(q0));

    wire is_state_A;
    c2 decode_state_a (.D00(1'b0), .D01(1'b1), .D10(1'b0),
.D11(1'b0),.A1(q1), .B1(1'b0), .A0(q0), .B0(1'b1), .out(is_state_A));
    wire is_state_B;
    c2 decode_state_b (.D00(1'b0), .D01(1'b0), .D10(1'b1),
.D11(1'b0),.A1(q1), .B1(1'b0), .A0(q0), .B0(1'b1), .out(is_state_B));
    c2 decode_state_c (.D00(1'b0), .D01(1'b0), .D10(1'b0),
.D11(1'b1),.A1(q1), .B1(1'b0), .A0(q0), .B0(1'b1), .out(done_rnd));

    assign load_shreg  = is_state_A;
    assign inz_counter = is_state_A;
    assign shift_shreg = is_state_B;
    assign en_counter  = is_state_B;

endmodule
```

**Final results:**

